🏡 Hearth

Consolidated Design Docs · Qwen Cloud Hackathon (EdgeAgent) · 2026-07-02

1. **Hearth — Overview** README.md
2. **Inventory** INVENTORY.md
3. **00 — Capabilities** docs/00-capabilities.md
4. **01 — Infra (Alibaba Cloud)** docs/01-infra-alibaba-cloud.md
5. **02 — Data Model** docs/02-data-model.md
6. **03 — Agent & MCP Surface** docs/03-agent-mcp-surface.md
7. **05 — UX / Product Design** docs/05-ux.md

README.md

🏠 Hearth

Describe your home in plain words. It figures out the rest.

Hearth is a home automation platform where you don't write rules — you *say what you want*, and an LLM agent configures your sensor nodes and reasons about live data to make it happen. Plug in a node, tell it "*warn me if the garage is open after dark and it's cold, and turn on the heater,*" and Hearth wires the sensors, the logic, and the actions for you — then reasons about the real world as it runs.

> **Global AI Hackathon Series with Qwen Cloud — Track 5: EdgeAgent** *(Hearth is a working name — trivially renamed.)*

The one-paragraph pitch

Home automation today makes you *program* it — rules, thresholds, YAML, IFTTT chains. That's why 99% of people never automate anything. Hearth replaces the rules with an agent. A cheap kit of ESP sensor/actuator nodes self-describes what it can see and do; a Raspberry Pi hub orchestrates them; and **Qwen Cloud** does the two things no rule engine can: it **turns your spoken intent into a working deployment**, and it **reasons about ambiguous, open-ended situations at runtime** (including *seeing*, via Qwen-VL). Buy the kit, plug in nodes, talk to your house.

Why this isn't Home Assistant (the test we hold every feature to)

> *Would this be meaningfully worse if we deleted Qwen and dropped in a 50-line script?*

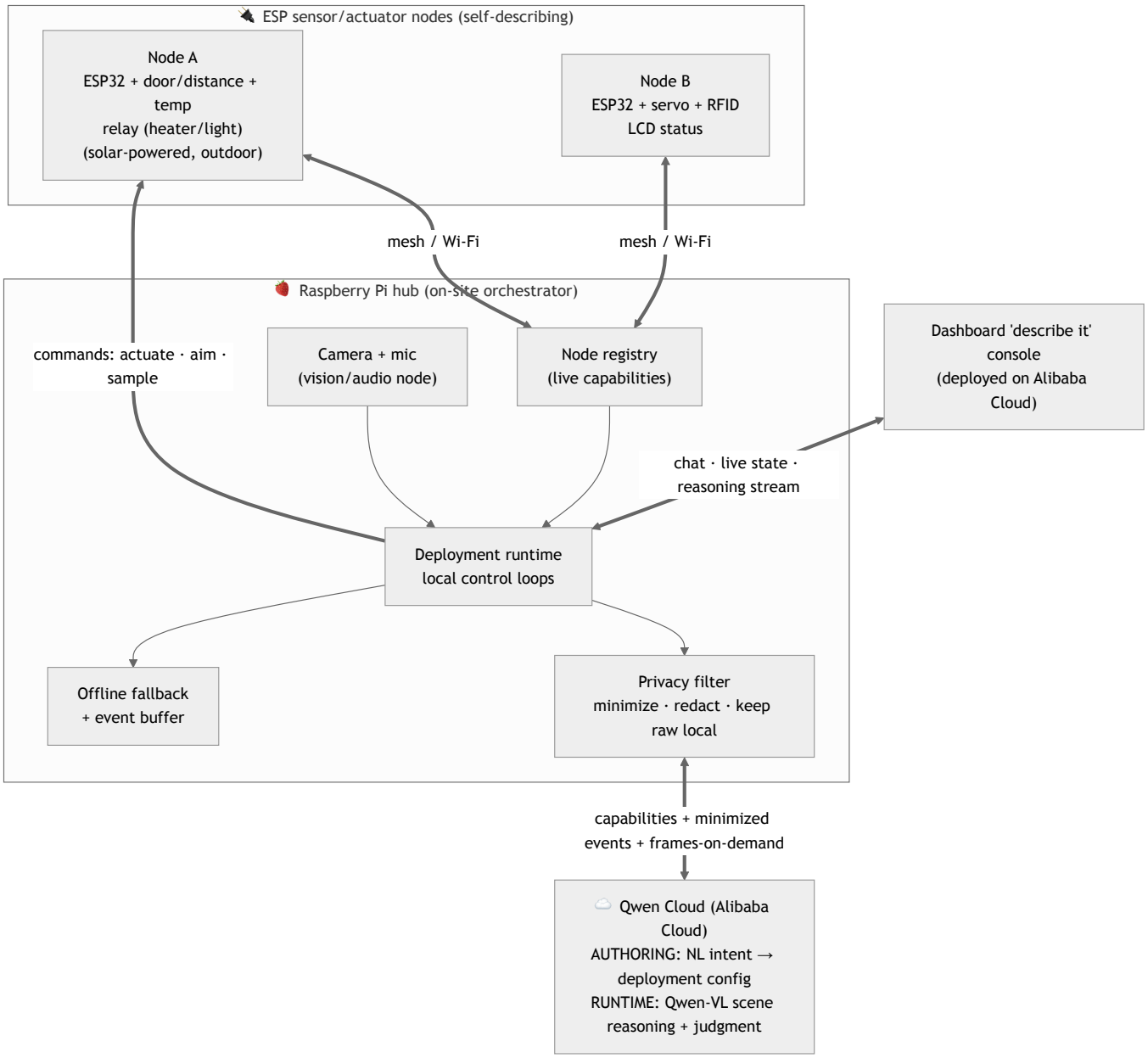
For a rule engine, no. For Hearth, **emphatically yes** — because Qwen is load-bearing in two distinct places a script can't touch:

1. Authoring time — natural language → a real deployment (program synthesis). You say "*let me know if someone who isn't a household member is at the front door.*" Qwen sees the available node capabilities (a camera here, a door sensor there, RFID tags for the family) and **synthesizes the deployment**: which sensors to bind, the trigger logic, the action, when to escalate. *You never wrote a rule.* A script can't turn an open-ended wish into a config.

2. Runtime — reasoning about the messy real world. At trigger time Qwen judges the actual situation instead of firing a dumb threshold: *is this person a household member? is this worth interrupting you for? what does this combination of signals actually mean?* With a camera node, **Qwen-VL understands the scene** — open-vocabulary perception that no local threshold can do.

Everything else (nodes, mesh, the hub) is honest plumbing. The intelligence lives exactly where only an LLM can provide it.

Architecture



Tier	Hardware	Job
Nodes	ESP32 + any sensors/actuators	Self-describe capabilities, report readings, execute commands, local safety veto
Hub	**Raspberry Pi** (+ camera/mic)	Node registry, run deployments, privacy filter, offline fallback, call Qwen
Cloud	**Qwen Cloud**	**Authoring** (NL→config) + **runtime reasoning / Qwen-VL** — the bulk

The agent loop runs on the **on-site hub**, so raw video/audio never leaves the house and the edge↔cloud orchestration is itself an edge activity.

How it maps to the Track 5 rubric

"*Qwen-powered physical devices that perceive via edge sensors, reason via cloud APIs/Skills, and act locally — with robust edge-cloud orchestration under bandwidth/latency constraints, privacy-aware data handling, and graceful degradation in offline/weak-network scenarios.*"

Rubric asks for…	Hearth delivers
perceive via edge sensors	Self-describing ESP nodes (temp, distance/door, RFID, motion) + hub camera/mic
reason via cloud APIs/Skills	Qwen does authoring (NL→config) **and** runtime scene/judgment reasoning — the bulk
act locally	Nodes drive relays/servos (heater, light, blinds, camera pan); local safety veto
orchestration under bandwidth/latency	Deployments run locally on the hub; only capabilities + minimized events + on-demand frames cross the link — never a stream
privacy-aware data handling	Raw video/audio stays on the hub; frames sent only on events, cropped/redacted; RFID identities hashed

Rubric asks for…	Hearth delivers
graceful degradation offline	Cloud is the normal mode; offline → the hub runs the last-authored deployment logic + local veto, buffers, and re-syncs on reconnect

The hero demo (≈3 min): *set up a house by talking*

1. **Describe it live.** "*Warn me if the garage door's open after dark and it's cold — and turn on the heater.*" → Qwen synthesizes the deployment from the node registry, live on screen.
2. **Trigger it.** Open the (real) door, drop the temp → the heater relay fires, phone pings.
3. **Add a vision deployment by voice.** "*Tell me if someone who isn't family is at the door.*" → camera node + Qwen-VL. Walk up → it reasons about the scene and identifies non-household.
4. **Kill the network.** The deployments keep running on the hub (fallback + veto); reconnect → Qwen posts a "here's what happened while you were offline" summary.
5. **Privacy reveal.** Side-by-side: the raw local frame vs the minimized/redacted payload sent to the cloud.

Kit & "hardware company" framing

Hearth is designed to ship as **preconfigured kits** (Starter: hub + 2 nodes; add-on nodes for door/climate/camera). The platform is general; kits are just curated bundles + a deployment library you can extend by *talking*. A BOM/cost breakdown ships with the submission to show the unit economics — this is a product, not a toy.

Hardware (the reference kit)

Raspberry Pi hub (+ USB webcam & mic) · 2× ESP32-WROOM-32 nodes · 2× nRF24L01 (node↔hub mesh; ESP-NOW/Wi-Fi fallback) · DHT11 temp/humidity · 2× HC-SR04 (door/distance) · servo (pan / blinds) · relay (heater/light) · RFID-RC522 (household identity) · I²C 16×2 LCD (node status) · solar panel (off-grid outdoor node) · GPS (optional, for outdoor/mobile nodes).

Scope discipline (how we don't drown in 8 days)

In: the node abstraction (self-describing) · hub registry + deployment runtime · **Qwen authoring (NL→config)** · **Qwen runtime reasoning + Qwen-VL** on the camera node · the "describe it" console · privacy filter · offline fallback · 2–3 live deployments · dashboard on Alibaba Cloud.

Breadth as narrative, not build: the wider "any sensor, any deployment" library is shown as config + a couple of genuinely working examples — we say so honestly rather than faking coverage.

Explicitly out (v1): manufacturing/packaging, on-device ML, a giant deployment catalog, multi-home cloud tenancy.

Ethics & privacy

A home camera/mic is highly sensitive. Raw media never leaves the hub; the cloud sees only event-driven, minimized, redacted data. Hearth *assists* — it is not a security or safety guarantee. We state this in the demo.

Status

🚧 Pre-build. This README is the locked project definition; implementation follows once it's solid. Track: **EdgeAgent**. Deadline: **2026-07-09**.

License

MIT (to be added).

Inventory — the Hearth reference kit

Everything on hand for the build. Items marked ⚠️ **confirm** are assumptions or missing specs — please correct.

Compute

Qty	Item	Notes	Role in Hearth
1	**Raspberry Pi** ⚠️ confirm model/RAM	Full Linux SBC; runs Python + Qwen SDK; SPI/I²C/UART/USB	**The hub** — node registry, deployment runtime, privacy filter, offline fallback, hosts camera/mic
2	**ESP32-WROOM-32** dev boards	LX6 dual-core @240 MHz, ~520 KB SRAM, Wi-Fi+BLE, ~4 MB flash	**Self-describing sensor/actuator nodes**
1	**Dev laptop** (macOS, Node 20 / Python 3.14 / Docker)	Build + flashing machine	Development, firmware upload, dashboard dev host

Rich perception (the hub's senses — makes Qwen-VL irreplaceable)

Qty	Item	Notes	Role
1	**USB webcam**	Plugs into the Pi	**Vision node** — frames sent to **Qwen-VL** on events (raw stays local). Guaranteed fallback camera
1	**USB microphone**	Plugs into the Pi	Audio events / voice interaction
1	**Insta360 X5**	360° camera; frames via USB-webcam-mode / RTMP live / periodic stills / SDK	**Optional hero "omni-vision" node** — whole-room awareness for Qwen-VL. ⚠️ integration risk; USB webcam is the fallback
1	**Sony α7 III (a7iii)**	Full-frame mirrorless; clean HDMI / USB webcam-mode (Imaging Edge)	**Demo-video production camera** (high-quality footage for the 3-min video) + optional high-fidelity vision node

Radios & connectivity

Qty	Item	Notes	Role
2	**nRF24L01**	2.4 GHz, SPI	Node↔hub link. ⚠️ multi-hop wants a 3rd radio; ESP-NOW/Wi-Fi is the fallback
—	ESP32 Wi-Fi / **ESP-NOW**	Built in	Alt node↔hub transport, no extra parts
—	Pi Wi-Fi/Ethernet	Built in	Hub uplink to Qwen Cloud

Node sensors

Qty	Item	Measures	Role in a deployment
2	**HC-SR04**	Ultrasonic distance	**Door open/closed**, presence, level
1	**DHT11**	Temp + humidity	Climate/comfort deployments (heater/fan triggers)
1	**RFID-RC522**	13.56 MHz RFID (SPI)	**Household-member identity** (UID hashed before upload)
1	**GY-NEO6MV2 GPS**	Position (UART)	Optional — outdoor/mobile nodes only

Node actuators & outputs

Qty	Item	Notes	Role
1	**Relay** ⚠️ confirm count/type	Switches mains-ish loads	**Heater / light / appliance** on-off
1	**Servo** ⚠️ confirm (SG90-class?)	PWM	Camera pan (active sensing), blinds/vent
1	**QAPASS 16×2 LCD + I²C**	HD44780 + PCF8574	On-node status readout

Power & storage

Qty	Item	Notes	Role
1	**Solar panel** (~60×55 mm)	Small PV	Off-grid outdoor node story. ⚠️ needs charge controller + battery + regulator — **do we have these, or is solar a demo prop?**
1	**microSD adapter**	SPI SD breakout	Optional node buffer (hub is primary buffer)
—	Pi microSD card	OS/storage	Hub OS + primary event buffer/log

Support parts — ⚠️ confirm on hand

Breadboard(s) · jumper wires (M-M / M-F) · **4.7–10 kΩ pull-up** for DHT11 · clean 3.3 V / decoupling for nRF24 · USB cables for flashing · external 5 V supply for servo/relay · header pins.

Cloud, software & accounts

Item	Status	Role
Qwen Cloud API key / hackathon credits	⚠️ **not yet — sign up ASAP**	Powers authoring + runtime reasoning (incl. Qwen-VL); blocks all interesting work
Qwen endpoint	`https://dashscope-intl.aliyuncs.com/compatible-mode/v1` (OpenAI-compatible)	qwen-plus / qwen-max / qwen-vl
Alibaba Cloud account	⚠️ confirm	Required for "Proof of Alibaba Cloud Deployment" (host the dashboard/console)
Qwen-Agent / Model Studio Skills	To evaluate	Deeper "used their stack" story
Dev tooling	✅ Node 20, Python 3.14, Docker, git	Build/deploy
Arduino IDE / PlatformIO	⚠️ not installed	ESP32 firmware build + flash

Environment & non-hardware assets

Asset	Role
The developer's own home	Live demo environment for the "describe it" hero demo
Senior full-stack developer (solo)	The team
~8 days (deadline 2026-07-09 14:00 PDT)	Timeline

Open questions this raises

1. **Raspberry Pi model + RAM?** (affects what runs locally, e.g. any local vision)
2. **Relay + servo — how many, what type?** (both are core actuators)
3. **Solar power chain** — panel alone won't run a node; do we have controller + battery + regulator, or is solar "for the photo"?
4. **3rd nRF24, or ESP-NOW/Wi-Fi** for node↔hub transport?
5. **Support parts** (DHT11 pull-up, nRF24 power, external 5 V) — on hand?

00 — Capabilities (neutral, tech-agnostic)

What the system *needs to be able to do*, described without naming any vendor or product. Stack choices live in `01-infra-alibaba-cloud.md` and are deliberately deferred until this list feels complete. If a capability here is wrong, fix it here first.

Global constraints that shape every choice

- **Scale-to-zero / cheap-at-idle** — a hobby-scale project on a **$40 credit** budget; idle cost must approach zero.
- **Intermittent edge link** — the on-site hub may lose connectivity; nothing may hard-depend on a live cloud link.
- **Privacy-first** — raw video/audio and identities stay local by default; only minimized, event-driven data leaves the home.
- **Judge-accessible without hardware** — the whole system must be demonstrable via a hosted build + a simulated home.
- **English, open-source, original work** (submission rules).

C1 — Reasoning (the brain): text + vision

- Run open-ended reasoning over structured context (the home model + recent data).
- **Authoring:** turn a natural-language intent into a structured deployment (which inputs, logic, action).
- **Runtime:** answer a bound question over live data, including **image understanding** (vision).
- Tool/function calling so the model can *act* through a defined interface.
- Constraint: token-frugal (few, event-driven calls) to survive the credit budget.

C2 — Backend compute (API + agent orchestration)

- Host the API the app and hub talk to; run the authoring + runtime reasoning orchestration.
- Event/HTTP triggered, **scales to zero**, no always-on VM.
- Runs our language of choice; modest CPU/memory (no model hosting — reasoning is a hosted API).

C3 — Source-of-truth store: the Home Model ("digital twin")

- Durable, authoritative record of homes → zones → devices → inputs → records → questions/runs → channels → context.
- Read/written by the app (authoring) and read by the hub (config pull).
- Document-shaped (nested JSON), low write volume, needs cheap-at-idle.

C4 — Observation store: readings, run results, timelines

- Append-heavy time-series of sensor samples + run answers/events (the "recorded" timelines).
- Query by device + time range for the app's scrubbable timeline.
- Retention policy (keep last N per record); cheap at rest.

C5 — Blob store: snapshots (images / audio clips)

- Store interval snapshots (JPEG frames, short audio) produced by Record policies.
- Serve them to the app via **time-limited, credential-free links** (so the backend stays thin).
- Lifecycle/expiry to control cost; privacy-scoped access.

C6 — Edge↔cloud sync (hub connectivity)

- Bidirectional, **tolerant of intermittent links**: config flows *down* to the hub, observations flow *up*.
- Buffer + reconcile after reconnect (no data loss, no duplicate actions).
- Lightweight on the hub; ideally a recognized device/twin model with offline shadow semantics.

C7 — Realtime push to the app

- Push live state, new run answers, and snapshot arrivals to the app without polling.
- Bidirectional or server→client is enough; must work behind mobile networks.

C8 — Secret / connection storage

- Securely store integration credentials (chat/SMS/email tokens, third-party device creds).
- Runtime retrieval (no secrets in code); rotation-friendly.
- Split: hub-local secrets on the hub; cloud-side secrets in a managed vault.

C9 — Outbound notification channels (Actions)

- Deliver an Action to the user: **mobile push, SMS, email, chat (e.g. Telegram)**.
- Pluggable — new channels are just adapters. Per-message billing acceptable.

C10 — Static hosting for the app's web build

- Serve the Expo **web** export (judge-accessible URL) over CDN, cheap at idle.

C11 — Identity & access (user ↔ home ↔ hub)

- Authenticate a user; authorize them to their home(s) and hub(s).
- Pair a physical hub to an account securely.
- For judging: a demo/guest path into a simulated home with no friction.

C12 — Observability

- Logs/metrics/traces for the backend and the agent's decisions (also feeds the audit trail).
- Enough to debug a live demo; nothing heavyweight.

Explicit non-goals (v1)

Multi-tenant scale, billing/monetization, high availability/DR, model *hosting* (we call a hosted model), broad third-party device integration (discover-and-list only), fleet management across many homes.

Open questions that affect capabilities (not stack)

1. Does the hub need to *fully* function offline (author new questions offline), or only *run* already-deployed ones offline? (Leaning: run-only offline; authoring needs the cloud brain.)
2. Is audio a v1 input or v2? (Affects C1/C5.)
3. Do we need per-user auth for v1, or is a single-home + guest-demo enough? (Affects C11.)

01 — Infra: Alibaba Cloud candidate stack

Maps each capability in `00-capabilities.md` to **candidate** Alibaba Cloud services. These are research-backed proposals with alternatives + tradeoffs. Everything here is chosen to be **scale-to-zero / cheap-at-idle** to survive the **$40 voucher**.

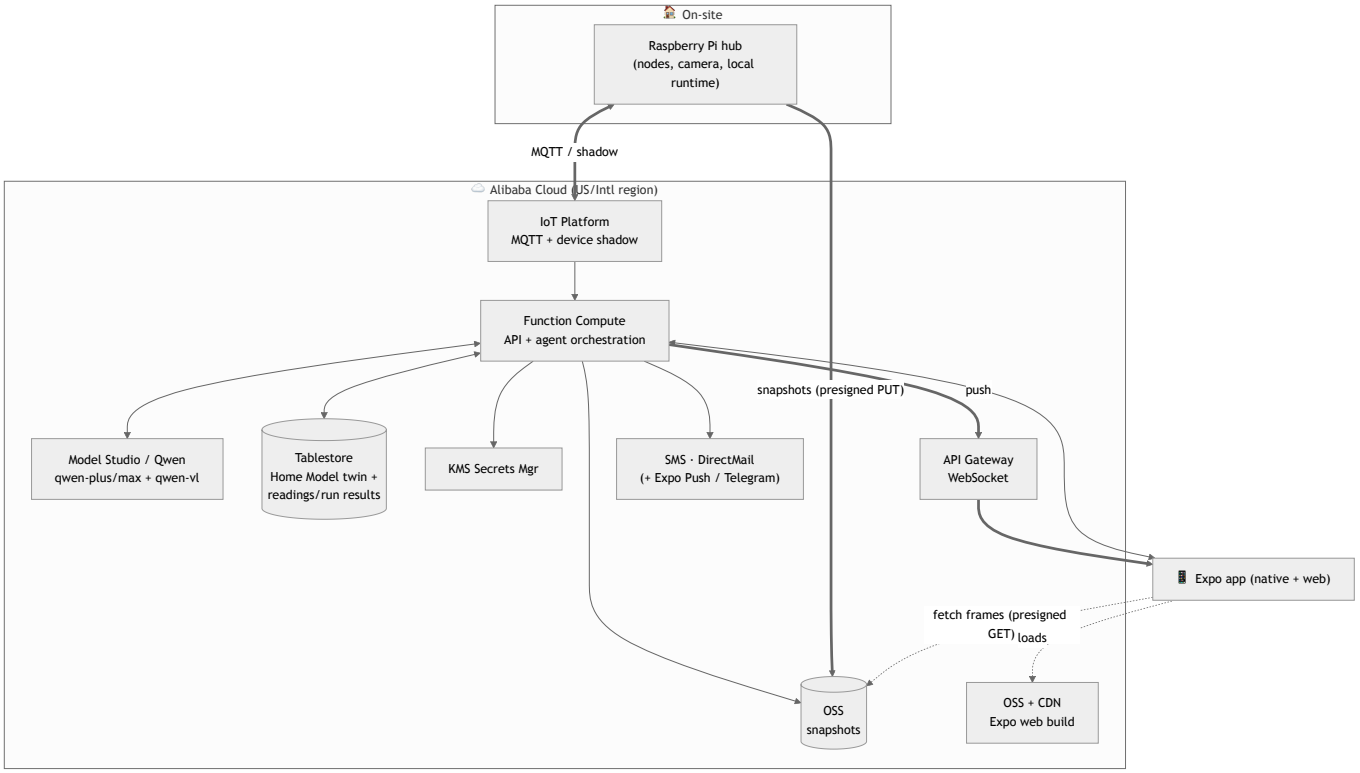
> **Standing principle: heavily favor Alibaba Cloud for every decision.** Breadth of *their* platform is worth judging points and reduces integration surface. Reach for an external service only when Alibaba has no reasonable equivalent (and say why).

> ⚠️ Account is **US-registered** → confirm whether we're on the **International** (`dashscope-intl.aliyuncs.com`) or **US/Virginia** (`dashscope-us.aliyuncs.com`) region and use the matching endpoints/consoles for *all* services. Verify exact free tiers/quotas at build time.

Capability → candidate service

#	Capability	Primary candidate	Alternative(s)	Scale-to-0?	Notes
C1	Reasoning (text+vision)	**Model Studio / DashScope — Qwen** (`qwen-plus` / `qwen-max`, **`qwen-vl`**), OpenAI-compatible	Qwen-Omni (audio)	pay-per-token	Required tech. Endpoint per account region.
C2	Backend compute	**Function Compute (FC)** — serverless, scales to zero	Serverless App Engine (SAE)	✅ yes	New-user free tier: ~1M invocations + 400k CU-s/mo. No model hosting needed.
C3	Home Model (twin)	**Tablestore** (serverless NoSQL, JSON)	ApsaraDB RDS/PolarDB **serverless** (SQL+JSONB)	✅ (set reserved CU=0 → pay-per-request)	Document-shaped twin; low write volume.
C4	Observation store	**Tablestore** time-series (same instance)	Lindorm (IoT TS), RDS	✅	Append-heavy readings + run results; query by device+time.
C5	Snapshots (blobs)	**OSS** + **presigned URLs**	—	✅ pay-per-use	App fetches frames directly via temp URLs → backend stays thin. Lifecycle rules for expiry.
C6	Edge↔cloud sync	**IoT Platform** (MQTT + **device shadow** = offline-tolerant twin)	DIY WebSocket (API Gateway + FC)	✅ pay-per-message; free 10 living devices/day	Shadow gives us the intermittent-link semantics for free. Adds setup. **Key decision — see §OD-1.**
C7	Realtime → app	**API Gateway WebSocket API** (native, on by default)	IoT Platform MQTT-over-WS; short polling	✅ pay-per-call	Push run answers/state to the app.
C8	Secrets	**KMS Secrets Manager** (`GetSecretValue` at runtime, rotation)	env in FC (weaker)	small per-secret	Cloud tokens (chat/SMS/email). Hub-local secrets stay encrypted on the hub.
C9	Notification channels	**SMS** (200+ countries) · **DirectMail** (email) · **Expo Push** (app) · Telegram (free)	Message Service (MNS) fan-out; Vonage (external)	pay-per-send	"All-on-Alibaba" = SMS+DirectMail; Expo Push is native to the app; Telegram = free demo garnish.
C10	Web hosting (Expo web)	**OSS static hosting + CDN**	FC + API Gateway	✅	Judge-accessible URL.
C11	Identity/access	**IDaaS (EIAM)**	DIY JWT in FC	✅	Or defer: single-home + guest-demo for v1 (see 00 OQ-3).
C12	Observability	**SLS (Log Service)**	FC built-in logs	✅ mostly	Also backs the decision/audit trail.

Candidate reference architecture



Cost / scale-to-zero read (fits $40)

- **Idle ≈ $0:** FC (pay-per-invoke), OSS (pay-per-use), Tablestore (reserved CU=0 → pay-per-request), API Gateway (pay-per-call), IoT Platform (pay-per-message, 10 free devices/day), SMS/DirectMail (pay-per-send).
- **The real spend is Qwen tokens.** Our **question-compilation (local predicates) + interval Records** deliberately minimize Qwen-VL calls → the architecture is token-frugal *by design*, which is exactly what the $40 budget requires. Good story for judges, real necessity for us.
- **Watch:** Tablestore reserved throughput (keep at 0), IoT Platform **Pro** device-management fees (Basic if we don't need them), any always-on component (avoid ECS entirely).

Judging leverage

Using **IoT Platform (shadow) + OSS + Tablestore + KMS + Qwen (VL + tool-calling)**, and exposing the home as an **MCP server** the Qwen agent calls, directly targets the rubric's repeated "**sophisticated use of Qwen Cloud APIs (custom skills, MCP integrations)**" and the mandatory "**Proof of Alibaba Cloud Deployment**." Breadth of *their* stack = points.

Open decisions (make once the problem is fully shaped)

- **OD-1 — Transport: ✅ DECIDED → Alibaba IoT Platform.** Hub = **gateway device**, nodes = **sub-devices**, live control + offline reconcile via **device shadow**. More "their stack" (scores), built-in intermittent-link twin semantics. DIY WebSocket rejected.
- **OD-2 — Store: Tablestore vs serverless RDS/PolarDB.** Tablestore = serverless NoSQL, ideal for the JSON twin + time-series, cheap-at-idle. RDS Postgres = familiar SQL + JSONB, easier ad-hoc queries. *Lean: Tablestore.*
- **OD-3 — Realtime: API Gateway WebSocket vs MQTT-over-WS vs polling.** *Lean: WebSocket; polling as the trivial fallback.*
- **OD-4 — Notifications: Alibaba-native (SMS/DirectMail) vs external (Expo Push/Telegram/Vonage).** *Lean: Expo Push (native to app) + Telegram (free) for the demo; add Alibaba SMS/DirectMail to strengthen the "all-on-Alibaba" claim.*
- **OD-5 — Auth: IDaaS vs DIY vs defer.** *Lean: defer to single-home + guest-demo for v1.*
- **OD-6 — Compute granularity:** one FC app vs several functions (authoring / runtime / sync / webhooks). *Lean: start as one, split if needed.*

Sources

- Function Compute (serverless, scale-to-zero, pricing): https://www.alibabacloud.com/en/product/function-compute/pricing · https://www.alibabacloud.com/help/en/functioncompute/fc/user-guide/introduction-to-serverless-gpus-1

- Qwen via Model Studio / DashScope (OpenAI-compat, Qwen-VL, endpoints): https://www.alibabacloud.com/help/en/model-studio/compatibility-of-openai-with-dashscope · https://www.alibabacloud.com/help/en/model-studio/qwen-vl-compatible-with-openai
- IoT Platform (device shadow, MQTT, pricing, 10 free devices/day): https://www.alibabacloud.com/help/en/iot/developer-reference/device-shadow · https://www.alibabacloud.com/product/iot/pricing
- Tablestore (serverless NoSQL, pay-as-you-go): https://www.alibabacloud.com/help/en/tablestore/what-are-billing-methods-and-items-of-tablestore
- API Gateway WebSocket: https://www.alibabacloud.com/help/en/api-gateway/cloud-native-api-gateway/user-guide/create-a-websocket-api
- OSS (presigned URLs, billing): https://www.alibabacloud.com/help/en/oss/user-guide/how-to-obtain-the-url-of-a-single-object-or-the-urls-of-multiple-objects · https://www.alibabacloud.com/help/en/oss/billing-overview
- KMS Secrets Manager: https://www.alibabacloud.com/help/en/kms/key-management-service/user-guide/secret-management-overview
- SMS / DirectMail: https://www.alibabacloud.com/help/en/sms/product-overview/what-is-alibaba-cloud-sms · https://www.alibabacloud.com/help/en/direct-mail/product-overview/directmail

02 — Data Model (single source of truth)

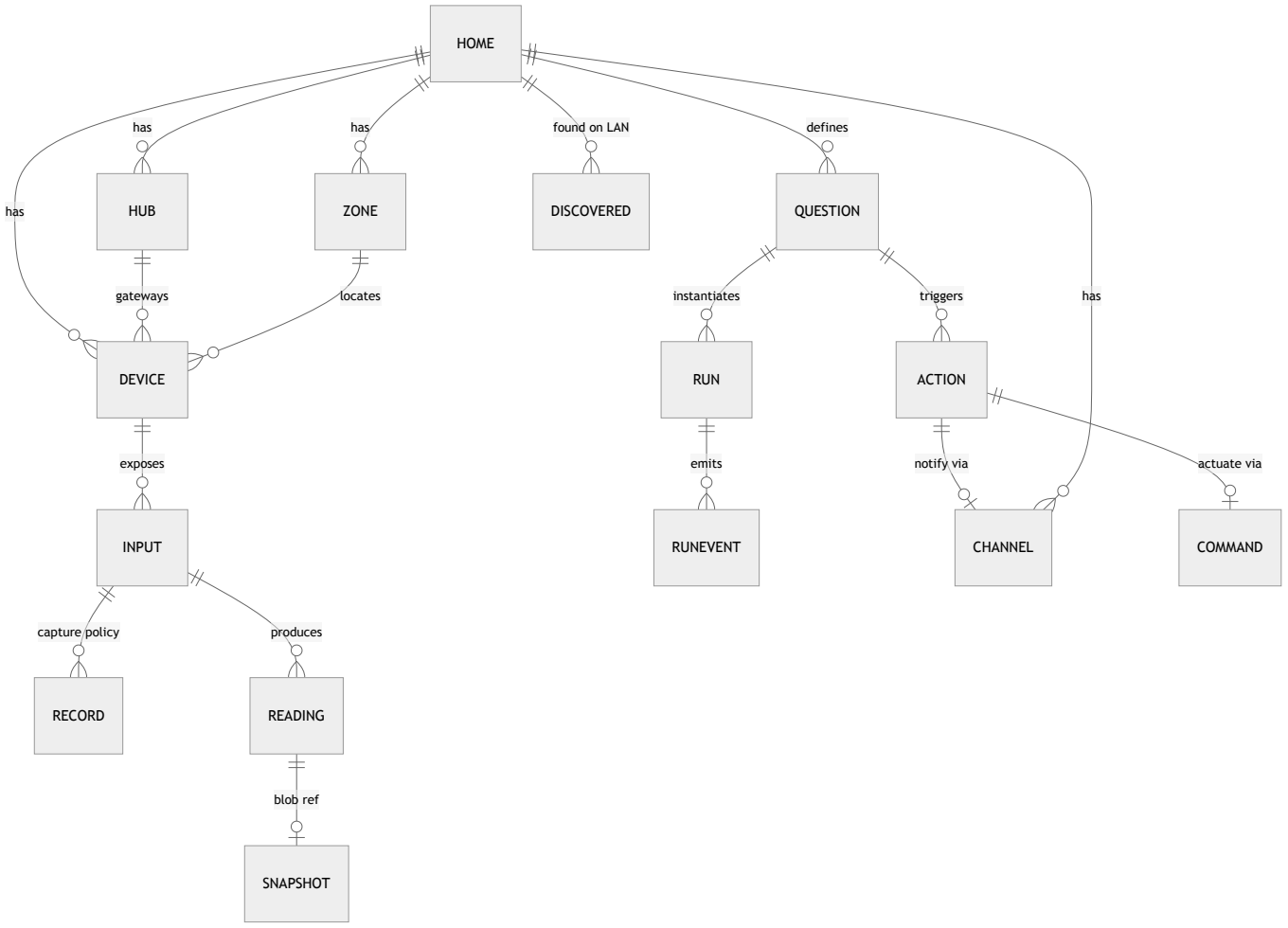
The complete data model for Hearth, designed solutions-architect style: entities, schemas, storage mapping, keys/TTL, and sync semantics. **Standing principle: heavily favor Alibaba Cloud.** OD-1 is settled → transport is **Alibaba IoT Platform** (gateway + sub-devices + device shadow).

Notation is language-neutral TypeScript-ish interfaces + JSON examples. This is a *model*, not code.

Design principles

1. **Type travels with the data.** Every datum is self-describing via a canonical `Type`, so the app and Qwen interpret any sensor without bespoke parsing.
2. **Config vs observation split.**
   - **Configuration** (the *Home Model twin*) — low-volume, authoritative, versioned.
   - **Observation** (readings, snapshots, run events) — high-volume, append-only, TTL'd.
3. **The twin is one versioned document per home.** This is the "source-of-truth object hosted in Alibaba Cloud" — small, cheap, optimistic-concurrency versioned.
4. **Live device state + offline reconcile = IoT Platform device shadow** (desired/reported). Large config is *pulled* by the hub; small live control goes through the shadow.
5. **Privacy by construction.** Raw blobs stay local unless a Record/Question needs them; identities (RFID) are hashed; only minimized, event-driven data is persisted to cloud.

Entity overview



- **Configuration (the twin):** Home, Zone, Hub, Device, Input, Record, Question, Action, Channel.

- **Observation (high-volume):** Reading, Snapshot, RunEvent, Command.
- **Transient:** DiscoveredDevice.

Canonical type system

`Type` is how a datum is interpreted. Shorthand id = `kind/unit` (e.g. `temperature/C`).

```
interface Type {
  kind: "temperature" | "humidity" | "distance" | "presence" | "switch"
      | "image" | "audio" | "rfid_uid" | "location" | "voltage" | "generic";
  dtype: "number" | "bool" | "enum" | "string" | "blob";
  unit?: string;          // "C", "%RH", "cm", "V" … (null for blobs)
  enum?: string[];        // for dtype "enum"
  range?: [number, number];
  mime?: string;          // blobs: "image/jpeg", "audio/wav"
  semantics?: string;     // human/LLM-readable meaning, e.g. "garage door open/closed"
}
```

Starter catalog: `temperature/C`, `humidity/%RH`, `distance/cm`, `presence/bool`, `switch/bool`, `voltage/V`, `rfid_uid/string` (hashed), `location/geo`, `image/jpeg`, `audio/wav`.

Configuration entities — the Home Model twin

Assembled into **one versioned document per home** (stored whole; also addressable per-entity via API).

```
interface Home {
  id: string;                 // "home_..."
  name: string;
  ownerId?: string;           // deferred; single-home + guest for v1
  createdAt: number;          // epoch ms UTC
  version: number;            // monotonic; optimistic concurrency
}

interface Zone { id: string; homeId: string; label: string; }

interface Hub {               // the Raspberry Pi — an IoT Platform GATEWAY device
  id: string;
  homeId: string;
  iot: { productKey: string; deviceName: string };   // IoT Platform identity
  status: "online" | "offline";
  lastSeen: number;
  fw?: string;
  appliedTwinVersion: number; // reported by hub via shadow (reconcile marker)
}

interface Device {            // an ESP node (IoT SUB-device) or a discovered/virtual device
  id: string;                 // "dev_..."
  homeId: string;
  hubId: string;
  zoneId?: string;
  kind: "hearth_node" | "third_party" | "virtual";   // virtual = simulator
  label: string;
  vendor?: string;            // from OUI/fingerprint or self-describe
  model?: string;
  transport: ("wifi" | "nrf24" | "usb" | "ip")[];
  iot?: { productKey: string; deviceName: string };   // sub-device identity (native nodes)
  status: "online" | "offline" | "discovered" | "ignored";
  lastSeen?: number;
  context?: DeviceContext;
```

```
  inputs: string[];           // inputIds
}

interface DeviceContext {     // powers the world model + semantic input resolution
  placementPhoto?: string;    // OSS key
  description?: string;       // Qwen-VL scene description or user text
  heading?: number;           // degrees, from phone compass
  location?: { lat: number; lon: number };   // coarse
}

interface Input {             // a typed datastream OR an actuator surface
  id: string;                 // "inp_..."
  deviceId: string;
  direction: "sensor" | "actuator";
  type: Type;
  defaultCadence?: string;    // Duration "5m","2s"
  writable?: boolean;         // actuators
  safety?: SafetyRule[];      // local veto constraints (enforced on the node)
}

interface SafetyRule {        // e.g. never open valve while leak latched
  when: string;               // predicate id / expression
  forbid: string;             // command/value forbidden
}

interface Record {            // configurable capture policy (interval snapshots etc.)
  id: string;                 // "rec_..."
  inputId: string;
  mode: "interval" | "on_event";
  interval?: string;          // Duration, e.g. "20s"
  retain: { count?: number; maxAge?: string };
  transform?: "raw" | "crop" | "downscale" | "redact";  // privacy/bandwidth
  active: boolean;
}

interface Channel {           // outbound notification connection (secret in KMS)
  id: string;                 // "chan_..."
  homeId: string;
  type: "expo_push" | "telegram" | "sms" | "email";
  config: Record<string, unknown>;   // non-secret (chatId, phone, email)
  secretRef: string;          // KMS Secrets Manager name for token/keys
}
```

Question, compilation, Action (the agentic core)

```
interface Question {
  id: string;                 // "q_..."
  homeId: string;
  text: string;               // the natural-language intent
  boundInputs: string[];      // inputIds Qwen selected
  compiledTo: "local" | "cloud_reason" | "cloud_vl";
  compiledSpec: LocalPredicate | CloudCheck;
  evalOn: "record" | "interval" | "event";  // cadence source (usually a Record)
  actions: string[];          // actionIds fired when answer flips true
  authoredBy: "qwen" | "user";
  createdAt: number;
  version: number;
}

// Compiled to run OFFLINE on the hub — the cheap, local, no-LLM path
```

```
interface LocalPredicate { expr: PredicateNode; }
type PredicateNode =
  | { op: ">"|">="|"<"|"<="|"=="|"!="; left: InputRef; right: number|string|boolean }
  | { op: "and"|"or"; nodes: PredicateNode[] }
  | { op: "changed"|"delta"; input: InputRef; window: string; threshold?: number };
interface InputRef { input: string; agg: "latest"|"mean"|"min"|"max"; window?: string; }

// Compiled to call Qwen in Function Compute — the open-ended / vision path
interface CloudCheck {
  model: "qwen-plus" | "qwen-max" | "qwen-vl";
  promptTemplate: string;
  send: { inputs: string[]; snapshots?: string[] };  // context to include
  outputSchema: object;       // JSON Schema for a structured answer
  maxCadence: string;         // rate-limit cloud calls (protects the $40 budget)
}

interface Action {
  id: string;                 // "act_..."
  homeId: string;
  type: "actuate" | "notify";
  // actuate:
  target?: { deviceId: string; inputId: string };   // actuator input
  command?: unknown;                                 // desired value
  // notify:
  channelId?: string;
  message?: string;           // template; Qwen may fill the specifics
}
```

Authoring flow: user NL intent → Qwen (in FC) reads the twin's device/input catalog → emits a `Question` with `boundInputs`, `compiledTo`, `compiledSpec`, sensible `Record` cadence, and `actions`. Qwen decides local-vs-cloud (§budget) — *admitting when the LLM isn't needed is a feature.*

Observation entities — high-volume, append-only

```
interface Reading {           // one sample from an input
  homeId: string; deviceId: string; inputId: string;
  ts: number;                 // epoch ms UTC
  type: string;               // "temperature/C"
  value?: number | boolean | string;   // scalar
  blobRef?: string;           // OSS key for image/audio
  buffered?: boolean;         // replayed from the hub's offline buffer
}

interface Snapshot {          // index row for a blob capture (blob lives in OSS)
  homeId: string; deviceId: string; inputId: string;
  ts: number;
  ossKey: string;             // "snap/{homeId}/{deviceId}/{inputId}/{ts}.jpg"
  meta?: { w?: number; h?: number; mime: string; bytes?: number };
  expiresAt?: number;         // mirror of OSS lifecycle
}

interface RunEvent {          // an evaluation result — the timeline + audit trail
  runId: string; homeId: string;
  ts: number;
  answer: { match: boolean; value?: unknown; confidence?: number };
  reasoning?: string;         // Qwen rationale (why) — audit
  evaluatedBy: "hub_local" | "qwen-plus" | "qwen-max" | "qwen-vl";
  contextRefs?: { readings?: number[]; snapshots?: string[] };
  actionsFired: string[];     // actionIds
```

```
}

interface Run {
  id: string;                 // "run_..."
  questionId: string; homeId: string;
  state: "active" | "paused";
  startedAt: number; lastEvalAt?: number;
  lastAnswer?: RunEvent["answer"];
}

interface Command {           // an actuation, realized via the device shadow desired-state
  id: string;                 // "cmd_..."
  homeId: string; deviceId: string; inputId: string;
  desired: unknown;
  issuedBy: string;           // runId or "user"
  status: "queued" | "sent" | "applied" | "vetoed";
  vetoReason?: string;        // from the node's local safety veto
  ts: number;
}
```

Transient — discovery

```
interface DiscoveredDevice {  // found on LAN, not yet connected (user must initiate)
  homeId: string;
  mac: string; vendor?: string;         // OUI lookup
  hostname?: string; ip?: string;
  services?: string[];                  // mDNS/SSDP service strings
  fingerprint?: Record<string, unknown>;
  suggestedType?: Type[];               // Qwen's guess from the fingerprint
  suggestedLabel?: string;
  status: "discovered" | "connected" | "ignored";
  seenAt: number;
}
```

Storage mapping (favor Alibaba)

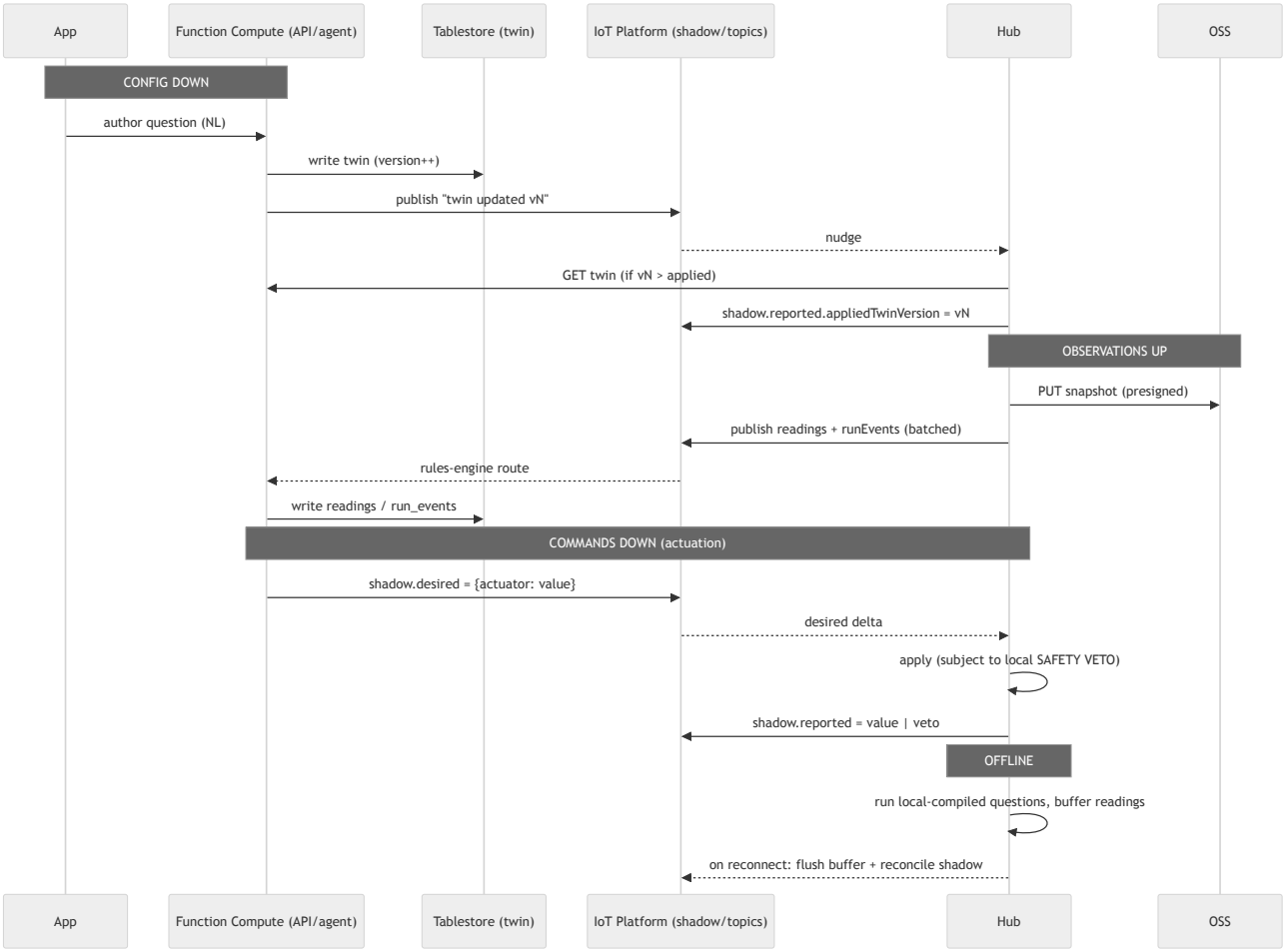
Entity	Alibaba service	Key design	TTL / lifecycle
Home Model twin (Home+Zone+Hub+Device+Input+Record+Question+Action+Channel refs)	**Tablestore** table `home_model`	PK = `homeId` → one JSON doc, `version` attr	none (small, versioned)
Reading (scalars)	**Tablestore** `readings`	PK = `homeId#deviceId#inputId`, SK = `ts`	TTL from `Record.retain`
Snapshot blob	**OSS**	`snap/{homeId}/{deviceId}/{inputId}/{ts}.jpg`	OSS lifecycle rule (expiry)
Snapshot index	**Tablestore** `snapshots`	PK = `homeId#deviceId#inputId`, SK = `ts`	TTL
RunEvent	**Tablestore** `run_events`	PK = `homeId#runId`, SK = `ts`	TTL
Run (live)	**Tablestore** `runs` (or in twin)	PK = `homeId`, SK = `runId`	none
Command log	**Tablestore** `commands` + **IoT shadow** desired	PK = `homeId#deviceId`, SK = `ts`	TTL on log
Device/Hub live state (online, appliedTwinVersion, actuator reported)	**IoT Platform device shadow**	per gateway/sub-device	n/a

Entity	Alibaba service	Key design	TTL / lifecycle
Channel secrets	**KMS Secrets Manager**	secret name in `Channel.secretRef`	rotation
DiscoveredDevice	**Tablestore** `discovered`	PK = `homeId`, SK = `mac`	short TTL

Tablestore notes: keep **reserved CU = 0** (pay-per-request); add a **Search Index** only if the app needs ad-hoc queries; use TTL for all observation tables to bound cost.

Sync & twin semantics (IoT Platform)

Hub = **gateway device**; each native node = **sub-device** under it.



- **Config down:** twin is authored in the cloud (Tablestore), version-bumped; hub pulls on nudge/boot and reports `appliedTwinVersion` via shadow → clean reconcile.
- **Observations up:** snapshots go **straight to OSS** via presigned PUT (thin backend); readings/run-events publish over IoT topics → rules engine → FC → Tablestore.
- **Commands down:** Actions set the sub-device **shadow desired**; the node applies subject to its **local safety veto** and reports back. Shadow reconciles if the node was offline.
- **Offline:** hub keeps running **local-compiled** questions + Records, buffers observations, and reconciles on reconnect. Cloud-compiled questions simply pause until the link returns.

Conventions

- **IDs:** typed prefix + opaque suffix — `home_`, `dev_`, `inp_`, `rec_`, `q_`, `run_`, `act_`, `chan_`, `cmd_`.
- **Timestamps:** `ts` = epoch **milliseconds, UTC**.
- **Durations:** short strings — `"2s"`, `"20s"`, `"5m"`, `"1h"`.
- **Versioning:** `Home.version` monotonic; writes are compare-and-set; hub reports applied version.

- **Privacy:** `rfid_uid` values are **hashed** before persistence; GPS coarsened; blob `transform` (crop/redact) applied at the hub before upload.

Open questions

1. Store `Run` inside the twin doc, or as its own table? (Lean: own table — runs churn; twin stays config-only.)
2. Do we need a Search Index on `readings` for the app timeline, or is a PK+time-range scan enough? (Lean: scan.)
3. Audio in v1? (Adds `audio/*` Types + OSS blobs + Qwen-Omni; lean v2.)
4. Multi-hub per home in v1? (Model supports it; demo likely single-hub.)

03 — Agent & MCP Surface

How Qwen reasons over the house and acts. The home is exposed as a **Home MCP server** the Qwen agent connects to — one typed interface to perceive and effect. This is the deliberate "**MCP integration / custom skills**" the rubric rewards. **Favor Alibaba:** the MCP server runs in **Function Compute**, fronts **Tablestore** (twin), **OSS** (snapshots), and the hub via **IoT Platform device shadow**; the agent is **Qwen** via Model Studio.

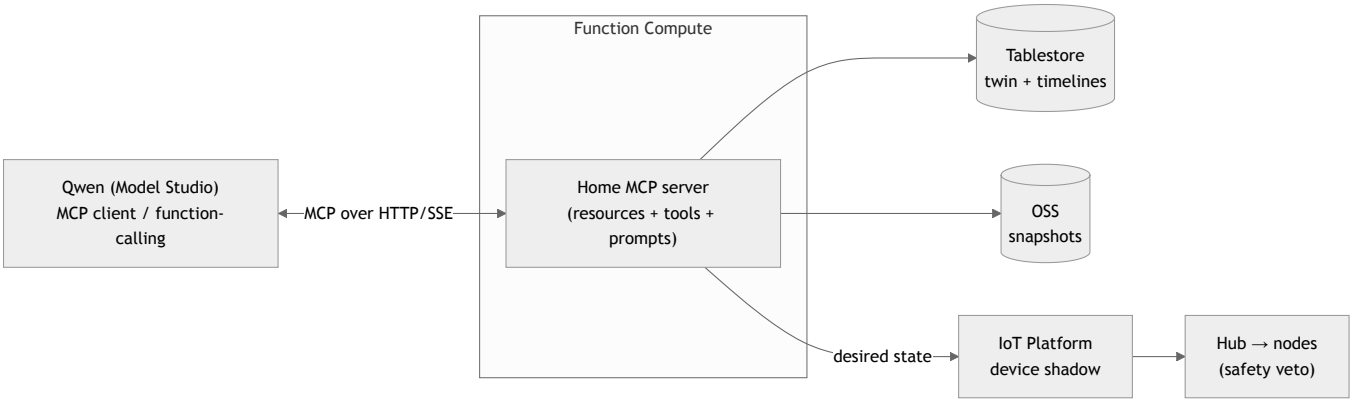
Depends on `02-data-model.md` (entities) — tools are typed operations over those entities.

Two modes, one surface (and a third path with no agent at all)

Mode	When	Agent does	LLM cost
Authoring	user writes a question / asks for suggestions	NL intent → a **compiled Question** (program synthesis)	one-shot, at design time
Runtime	a *cloud-compiled* question fires on its Record cadence	evaluate over fresh context, optionally **investigate**, decide actions	per eval, rate-limited
Local (no agent)	a *local-compiled* question	hub evaluates a predicate — **zero tools, zero LLM, offline**	none

The local path is the majority and the point of question-compilation: the agent is invoked only where open-ended reasoning is actually needed.

Architecture



Transport: MCP over HTTP/SSE, hosted in FC. If Model Studio's MCP *client* support is limited, the identical tool schemas are used via **Qwen function-calling** — so the catalog is transport-agnostic (MCP is the target, function-calling the fallback).

MCP primitives: resources vs tools vs prompts (idiomatic MCP)

- **Resources** (readable context, no side-effects): `home://model` (the twin/world-model), `home://inputs`, `home://readings/{inputId}`. The agent *reads* these to ground itself.
- **Tools** (effectful or computed operations): the catalog below.
- **Prompts** (server-provided templates): `authoring`, `runtime-eval`, `suggest-runs` — the reusable system prompts, versioned with the server.

Tool catalog (composable primitives, not a bespoke sprawl)

Generality comes from typed inputs + composition. ~11 tools cover the house.

Tool	Purpose	Availability
`describe_home()`	World-model summary (zones, devices, inputs, placements)	authoring, runtime
`list_inputs(filter?)`	Inputs with type + semantics	authoring
`read_input(inputId, agg?, window?)`	Latest/aggregated scalar reading	authoring, runtime
`query_history(inputId, from, to)`	Time-series slice (timeline)	authoring, runtime
`get_snapshot(inputId, at?)`	Fetch an image for Qwen-VL (OSS presigned)	authoring, runtime
`capture_now(inputId)`	Force a fresh snapshot — **active sensing**	runtime
`aim_sensor(deviceId, angle, reason)`	Move servo to look closer — **active sensing**	runtime

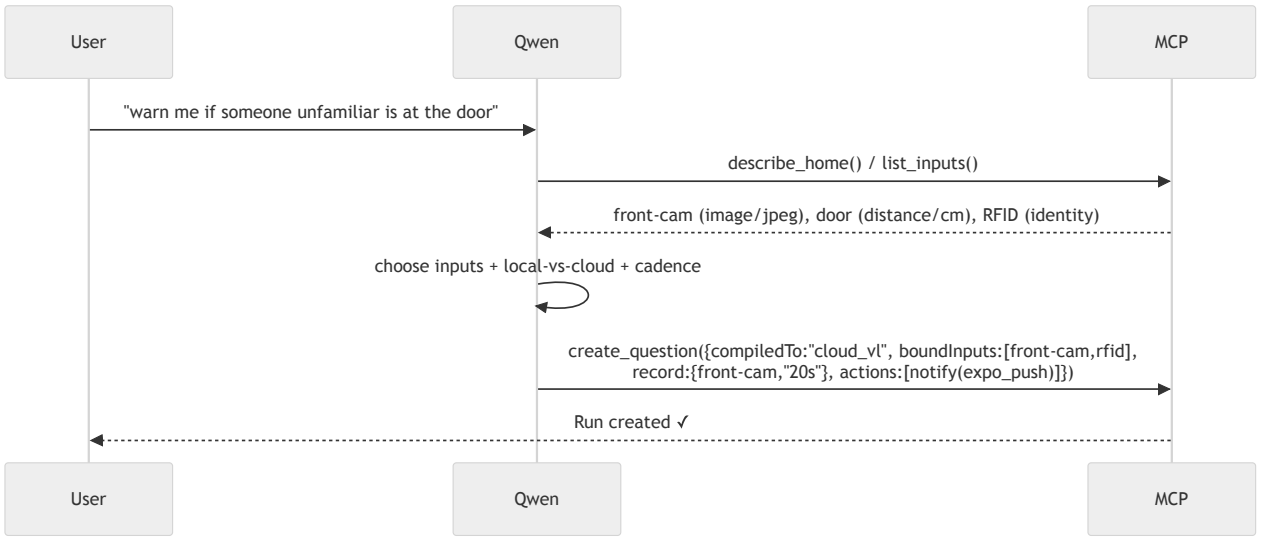
Tool	Purpose	Availability
`actuate(inputId, value, reason)`	Command an actuator (→ shadow desired, veto-gated)	runtime
`notify(channelId, message)`	Send an Action via a channel	runtime
`set_record(inputId, policy)`	Create/adjust a capture policy	authoring
`create_question(spec)`	Persist a compiled Question	authoring
`suggest_runs()`	Propose useful questions from the world model	authoring

Runtime is deliberately constrained: the runtime agent may *perceive* and *investigate* (`read_*` , `get_snapshot` , `capture_now` , `aim_sensor`) but may only fire the **pre-authored actions** of its Question — it decides *whether*, not *what*. Authoring holds the creative latitude (`create_question` , `set_record` , `suggest_runs`); runtime cannot invent new effects. Predictable + safe.

Representative schemas

```
// read
read_input(input: string, agg?: "latest"|"mean"|"min"|"max", window?: string)
  -> { ts: number, type: string, value: number|boolean|string }

get_snapshot(input: string, at?: number)
  -> { ossUrl: string, ts: number, mime: string }        // presigned; fed to Qwen-VL

// investigate (active sensing)
aim_sensor(deviceId: string, angle: number, reason: string) -> { ok: boolean }
capture_now(input: string) -> { ossUrl: string, ts: number }

// act (veto-gated, logged with rationale)
actuate(input: string, value: unknown, reason: string)
  -> { status: "sent"|"applied"|"vetoed", vetoReason?: string }
notify(channelId: string, message: string) -> { ok: boolean }

// author (program synthesis output)
create_question(spec: {
  text: string, boundInputs: string[],
  compiledTo: "local"|"cloud_reason"|"cloud_vl",
  compiledSpec: LocalPredicate | CloudCheck,
  evalOn: "record"|"interval"|"event",
  record?: { inputId: string, interval: string },
  actions: Action[]
}) -> { questionId: string }
```

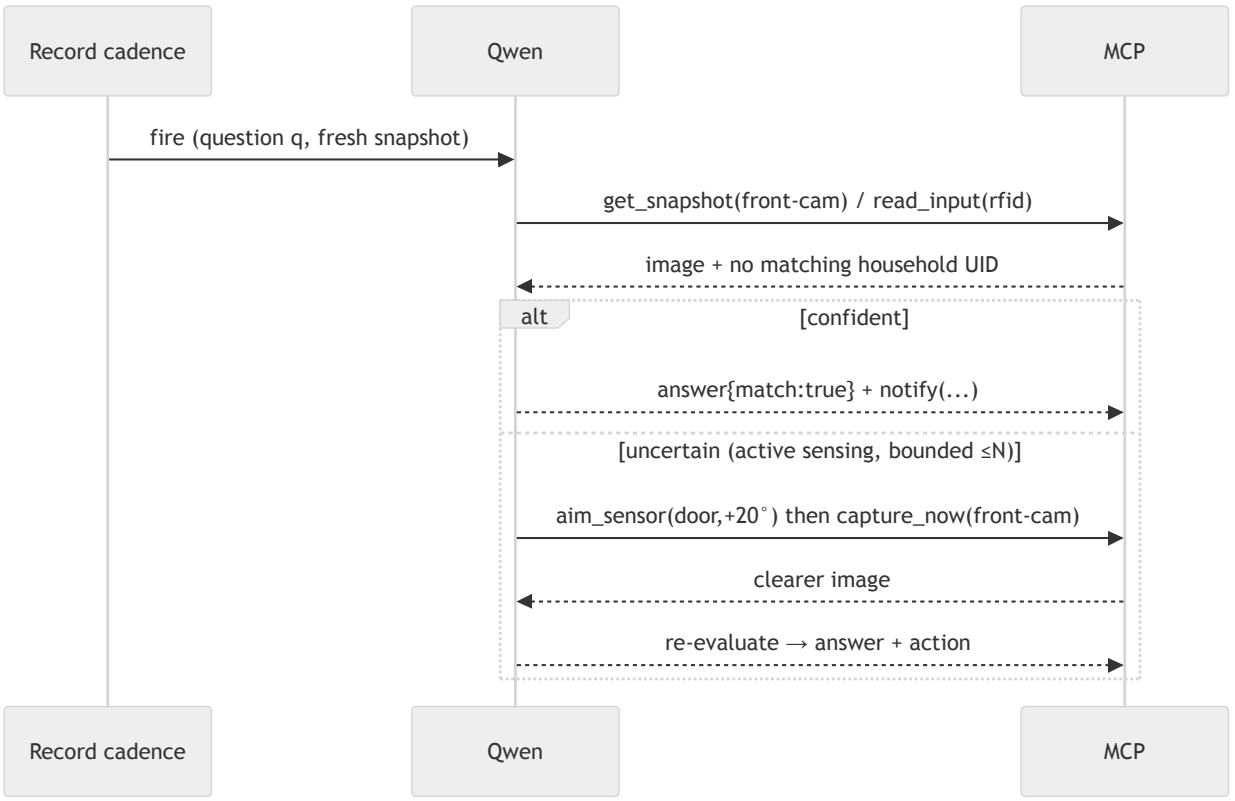
Authoring loop (NL intent → compiled Question)



Local-vs-cloud decision policy (the agent applies at authoring):

- Threshold/boolean over scalar inputs → `local` predicate (offline, no LLM).
- Needs image understanding → `cloud_vl`.
- Needs open-ended judgment/fusion over non-visual signals → `cloud_reason`.
- Always set the cheapest `maxCadence` that satisfies the intent (budget guard).

Runtime loop (cloud-compiled eval + active sensing)



The **active-sensing loop is bounded** (max N investigate iterations) so it can't spin or burn budget. This bounded "look closer when unsure" is the signature agentic behavior.

Guardrails

- **Safety veto** — every `actuate` is a shadow *desired* the node may refuse; the tool returns `vetoed` + reason. The agent never bypasses local safety.
- **Constrained runtime action space** — runtime can only fire the Question's authored actions.
- **Budget** — `CloudCheck.maxCadence` rate-limits evals; active-sensing bounded to N; local questions cost nothing. All Qwen calls are logged as `RunEvent.reasoning` (audit + explainability).
- **Idempotency** — `actuate` / `notify` carry the `runId+ts` so repeats (retries, reconnect replays) don't double-fire.

Tool → data-model mapping (ties to 02)

Tool	Operation
`read_input` / `query_history`	read `Reading` (Tablestore)
`get_snapshot` / `capture_now`	`Snapshot` + OSS presigned GET (capture = request hub grab)
`aim_sensor` / `actuate`	write `Command` → IoT shadow *desired*; node reports *reported*/veto
`set_record`	upsert `Record` in the twin
`create_question`	upsert `Question` (+ compiledSpec) in the twin, version++
`notify`	resolve `Channel` (+ KMS secret) → SMS/DirectMail/Expo/Telegram
`describe_home` / `list_inputs`	read the twin (`home://model`)

Open questions

1. MCP client support in Model Studio today, or function-calling fallback for v1? (Design is either.)
2. Does `capture_now` push a shadow request to the hub and await the frame (sync), or return the next Record frame (async)? (Lean: short-await with timeout → fall back to latest.)
3. Should `suggest_runs` run proactively on onboarding, or only on demand? (Lean: on onboarding — it's a hero beat.)
4. One MCP server, or split perception vs actuation servers? (Lean: one for v1.)

05 — UX / Product Design

How Hearth looks and feels — for the **homeowner** (real use) and the **judge** (must grok the value in 3 minutes, with no hardware). The guiding idea: **make the agent's reasoning visible.** The thinking *is* the product and the wow; hide it and we're just another sensor app.

Assumptions I'm designing against (unanswered questions — override anytime)

- **A1 Vision** = lightweight **local motion trigger** → **Qwen-VL** (latency + $40 budget). Fixed-interval VL is too slow/costly.
- **A2 User-defined detections** = **open-vocabulary** (`cloud_vl`), plain-language target, no training, no fixed object list.
- **A3 Active sensing** ("look closer") is in the UX; it depends on a **pan/tilt camera rig** decision (TBD). Shown as a first-class moment.
- **A4 "Describe it"** = **voice via the phone's native speech-to-text** → **text to Qwen** (no backend audio pipeline), plus typing. Audio understanding = v2.
- **A5 Judge access** = hosted **Expo Web** against a **simulated home** replaying canned real images/telemetry (so Qwen-VL sees real frames, zero hardware).
- **A6 Scope** = discover-and-list third-party + integrate the USB webcam; audio input v2.

Design principles

1. **Show the reasoning.** Every run answer carries Qwen's plain-language "why." Tap-to-expand.
2. **Plain language in, plain language out.** No rules, no YAML, no thresholds surfaced.
3. **Two audiences, one app.** The homeowner flow *is* the judge flow — no separate "demo mode" UI, just a simulated home behind it.
4. **Progressive disclosure.** Calm surface; depth on tap (the compiled spec, the snapshot, the trace).
5. **Privacy is visible.** A persistent "raw stays local" signal; a raw-vs-sent reveal.
6. **The house is a first-class object.** The world model (rooms/devices) is something you *see*, not a hidden config.

Information architecture

Expo tab bar: **Home · Describe · Inputs · Activity · Settings**, plus a one-time **Onboarding** flow.

```
Onboarding ─▶ Home ─▶ Run detail
                │
     ┌──────────┬──────────┬──────────┐
   Describe    Inputs     Activity     Settings
 (author Q)  (devices→   (alerts/     (hub, channels,
             inputs→      feed)         privacy)
             records)
```

Screen 1 — Onboarding & discovery (first "it understood my house" beat)

```
┌──────────────────────────────────┐
│  Setting up your hub…            │
│  ✓ Found 5 devices on your LAN   │
│                                  │
│  CONNECT (you choose)            │
│  ◉ Garage Node · Hearth  [add]   │
│  ◉ Front Cam   · Hearth  [add]   │
│  ○ Sonos One   · listed          │
│  ○ HP Printer  · listed          │
│                                  │
│  ── after add: give context ──   │
│  📷 snap where it's placed       │
│  Qwen-VL: "above garage door,    │
│   facing the driveway"   [ok]    │
│                                  │
```

```
|  ▸ "Here's what I understand   |
|     about your home…" (summary)|
|  ▸ Suggested run: watch for    |
|     deliveries?         [try]  |
└────────────────────────────────┘
```

- Discovery is **passive**; connecting is **user-initiated** (consent model from doc 02).
- The **house-explains-itself summary + suggested runs** is the payoff of the world model — a hero beat.
- States: scanning / found / nothing-found / permission-needed (camera for placement photo).

Screen 2 — Home (world model + active runs)

```
┌────────────────────────────────┐
|  Hearth              ◐ online  |
|  "3 things watching your home" |
|                                |
|  ▸ ACTIVE RUNS                 |
|  ┌────────────┐ ┌─────────────┐ |
|  |Front door | |Garage cold | |
|  |unfamiliar?| |+ open?     | |
|  |  ✓ clear  | |  ⚠ acted   | |
|  └────────────┘ └─────────────┘ |
|                                |
|  ▸ YOUR HOME                   |
|  Entry    ◉cam ◉door           |
|  Garage   ◉temp ◉relay         |
|  Nursery  ◉temp                |
|                                |
|  [ + Describe something new ]  |
└────────────────────────────────┘
```

- At-a-glance run status (clear / watching / acted). The world model is browsable by zone.
- Online/offline pill is honest and prominent (ties to graceful degradation).

Screen 3 — Describe-it console (the hero screen)

```
┌────────────────────────────────┐
|  Describe                      |
|  What should your home watch?  |
|  ┌──────────────────────────┐ |
|  | "tell me if someone      | |
|  |  unfamiliar is at the door"| |  ← type or 🎤 speak
|  └──────────────────────────┘ |
|                          [🎤][→]|
|                                |
|  ── Qwen compiled this ──      |
|  👁  Watches: Front Cam, RFID  |
|  🧠  Reasoning: Cloud (Qwen-VL)|
|      on motion, ≤ every 20s    |
|  🔔  Action: push-notify you   |
|  💸  Est. cost: low            |
|  🔒  Raw video stays local     |
|                                |
|  [ Looks right — start Run ]   |
|  [ Adjust ]   [ Explain ]      |
└────────────────────────────────┘
```

- The magic is **showing the compiled Question** (inputs, local-vs-cloud, cadence, action, cost, privacy) *before* it runs — proof the agent understood, and transparency.
- "Adjust" lets you refine in language; "Explain" shows why it chose those inputs.
- This screen carries the whole thesis: *describe → it configures.*

Screen 4 — Input Manager (devices → typed inputs → records)

```
┌──────────────────────────────┐
│ Inputs                       │
│ ▸ Front Cam (Entry)   ◉online │
│    • image/jpeg  [Record 20s] │
│    🌄🌄🌄🌄  (snapshot filmstrip)│
│ ▸ Garage Node (Garage)       │
│    • temperature/C  6.4°C  ▁▃ │
│    • distance/cm    312cm    │
│    • switch (heater) ○ off   │
│ ▸ Front Door                 │
│    • distance/cm  door: closed │
└──────────────────────────────┘
```

- Every input shows its **type** (the "type travels with data" idea, made visible), latest value, a sparkline, and its Record policy (tap to change cadence/retention).
- Actuators show state + are manually togglable (with the same safety veto).

Screen 5 — Run detail / timeline (the proof it's a real agent)

```
┌──────────────────────────────┐
│ ← Front door: unfamiliar?    │
│   Cloud · Qwen-VL · active   │
│                              │
│  Timeline   [────●──]  scrub │
│  🌄🌄🌄🌄🌄🌄                  │
│           ▲ 14:02 flagged    │
│                              │
│  14:02 · ⚠ MATCH             │
│  "Person at the door isn't in │
│   your household set. The first│
│   frame was unclear, so I    │
│   panned to get the face."   │
│   ↳ looked closer (aim +20°) │  ← active-sensing trace
│   ↳ notified you (push)      │
│  [ view snapshot ]  [ why? ] │
│                              │
│  🔒 raw frame never left home │
└──────────────────────────────┘
```

- The **reasoning trace + active-sensing steps** are the screen judges will remember. This is where "it's an agent, not a sensor" is *shown*, not claimed.
- Scrubbable **filmstrip** (Records) with event markers; tap an event → snapshot + full rationale.

Screen 6 — Activity / alerts feed

Chronological stream of what fired across all runs, each with a one-line reason and a tap-through to the run event. This is also where the **"while you were dark" reconnect summary** appears after an offline gap.

Screen 7 — Settings

Hub status + `appliedTwinVersion`, channels (Expo push / Telegram / SMS / email + which secret), privacy controls (what's allowed to leave the home), and the simulated-home toggle (for judges/dev).

Signature moments → where they live

Moment	Screen	Why it lands
"Describe it → compiled"	Describe console	The thesis, shown before it runs
"It understood my house"	Onboarding summary + suggested runs	World model pays off
"It looked closer" (active sensing)	Run detail trace	Agent ≠ sensor, *shown*
"While you were dark"	Activity feed	Graceful degradation, visible
Privacy reveal (raw vs sent)	Run detail / Settings	Rubric's privacy clause, made concrete

The 3-minute demo, screen by screen

1. **Onboarding** — hub finds & names LAN devices; add cam + garage node; snap placement; Qwen describes the home + suggests a run. *(0:00–0:45)*
2. **Describe console** — speak "*tell me if someone unfamiliar is at the door*"; watch Qwen compile it (inputs, Qwen-VL, action, privacy). Start the run. *(0:45–1:30)*
3. **Trigger + Run detail** — someone approaches; the run flags a match; the trace shows "*I panned to confirm the face*" → push fires on the phone. *(1:30–2:20)*
4. **Offline beat** — cut the uplink; a local-compiled run (garage cold+open) still fires; reconnect → Activity shows the "while you were dark" summary. *(2:20–2:45)*
5. **Privacy reveal** — raw local frame vs the minimized payload that went to cloud. *(2:45–3:00)*

Visual direction (proposed — swappable)

- **"Warm-precise."** Hearth = home warmth, but the app must read as *credible/agentic*. Calm neutral surfaces, one **ember accent**, generous whitespace, rounded cards, clear typographic hierarchy.
- **Reasoning motif** — the agent's "why" gets a distinct, quiet styling (a left-rule "trace" block with step ↳ markers) so thinking is recognizable throughout.
- **Semantic status** — *clear* / *watching* / *acted* / *offline* each get a consistent color+icon (not locked here).
- **Light + dark** both first-class.
- **Data-viz elements** (Home run cards, Input sparklines, Run filmstrip/timeline): when we build these, apply our **dataviz methodology** for a validated, accessible palette (light/dark) — deferred to implementation, not guessed here.

Component inventory (reusable)

`DeviceCard` · `InputChip` (shows Type) · `Sparkline` · `RunCard` (status) · `ReasoningTrace` · `Filmstrip + SnapshotViewer` · `CompiledQuestionPreview` · `StatusPill` (online/offline) · `PrivacyBadge` · `ChannelPicker` · `SuggestedRunCard`.

Cross-cutting states

- **First-run / empty:** no devices → guide to plug in the hub; no runs → nudge to Describe.
- **Offline:** global pill; runs show which are still evaluating locally vs paused (cloud).
- **Loading:** skeletons on cards; the Describe compile shows a short "thinking" state.
- **Error:** Qwen unreachable → "reasoning paused, local watches still active"; actuation vetoed → show the veto reason.

Accessibility

Semantic status never color-only (icon + label); large tap targets; VoiceOver labels on cards; captions on any demo audio; respects reduced-motion.

Open questions (still yours to call)

1. **Hero screen** — Describe console vs Run-detail-with-reasoning as *the* memorable one? (I've built both; leaning Describe as the opener, Run detail as the closer.)
2. **Aesthetic** — is "warm-precise Hearth" right, or do you want control-room/technical, or iOS-clean?
3. **Active-sensing rig** — confirm the pan/tilt camera so Screen 5's headline trace is physically real.
4. **Voice-first or type-first** for Describe (A4)?